RAG-Based Customer Support Assistant
Low-Level Design (LLD)

1. Module-Level Design

The system is divided into multiple modules, where each module performs a specific function in the RAG pipeline. These modules work together to process documents, retrieve relevant information, and generate accurate responses.

- **Document Processing Module**

  This module is responsible for loading the PDF document using PyPDFLoader.

  It extracts text data from the document and prepares it for further processing.

- **Chunking Module**

  This module splits the document into smaller chunks using RecursiveCharacterTextSplitter.

  Chunking helps in better context management and improves retrieval accuracy.

- **Embedding Module**

  This module converts text chunks into vector embeddings using HuggingFace embeddings.

  These embeddings represent semantic meaning of the text.

- **Vector Storage Module**

  This module stores embeddings in ChromaDB.

  It acts as a vector database that supports efficient similarity search.

- **Retrieval Module**

  This module retrieves relevant chunks based on user queries.

  It compares query embeddings with stored embeddings and returns the most relevant results.

- **Query Processing Module**

  This module takes user input and prepares it for retrieval and response generation.

  It ensures the query is properly formatted and processed.

- **Graph Execution Module**

  This module uses LangGraph to control the workflow of the system.

It manages nodes and edges to ensure proper execution flow.

- **HITL Module**

This module handles escalation to human support.

If the system cannot generate a confident response, the query is forwarded to a human.

2. Data Structures

The system uses different data structures to represent documents, chunks, embeddings, and query-response interactions. These structures help in organizing data efficiently throughout the pipeline.

- **Document Representation**

The document is represented as a collection of text extracted from the PDF.

Each page of the PDF is stored as a separate document object.

- **Chunk Format**

Each document is divided into smaller chunks.

A chunk contains a portion of text along with metadata such as page number or position.

- **Embedding Structure**

Embeddings are numerical vectors generated from text chunks.

These vectors represent semantic meaning and are stored in the vector database.

- **Query-Response Schema**

The query-response schema defines how user input and system output are structured.

The input is a text query, and the output is a generated response based on retrieved context.

- **State Object for Graph**

The state object is used in LangGraph to store and pass data between nodes.

It contains fields such as query, response, and final output, ensuring smooth workflow execution.

3. Workflow Design

The system uses LangGraph to define the workflow of execution.

It organizes the processing steps into nodes and connects them using edges to control the flow of data.

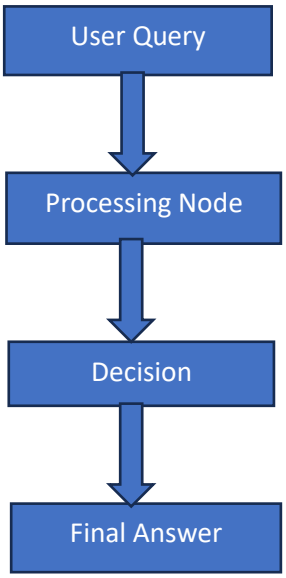

Figure: Workflow Design using LangGraph for RAG-Based System

- **Nodes**

The workflow consists of two main nodes:

1. Processing Node:

This node handles retrieval and response generation.

It takes the user query, retrieves relevant chunks, and generates a response using the LLM.

2. Output Node:

This node is responsible for producing the final output.

It determines whether to return the generated answer or escalate the query.

- **Edges (Flow Between Nodes)**

Edges define the flow between nodes.

In this system, the flow starts from the processing node and moves to the output node, ensuring sequential execution.

- **State (Data Flowing Between Nodes)**

The state object carries data between nodes.

It includes fields such as query, response, and final output.

Each node updates the state, and the updated data is passed to the next node.

4. Conditional Routing Logic

Conditional routing logic is used to decide how the system should respond to a user query.

Based on certain conditions, the system either generates an answer or escalates the query to human support.

- **Answer Generation Criteria**

  The system generates an answer when relevant document chunks are successfully retrieved and the LLM produces a meaningful response based on the context.

- **Escalation Criteria**

  The system escalates the query to human support when it is unable to generate a reliable or sufficient response.

- **Low Confidence**

  If the generated response is too short or lacks clarity, it is considered low confidence and is escalated.

- **Missing Context**

  If no relevant chunks are retrieved from the vector database, the system cannot answer the query and escalates it.

- **Complex Query**

  If the query is too complex or outside the scope of the document, the system forwards it to human support.

5. HITL Design

The Human-in-the-Loop (HITL) design allows the system to involve human support when the AI is unable to provide a reliable response.

This ensures better accuracy and improves user trust in the system.

- **When Escalation is Triggered**

  Escalation is triggered when the system fails to retrieve relevant information, generates a low-confidence response, or encounters a complex query beyond its capability.

- **What Happens After Escalation**

  Once escalation is triggered, the system notifies that the query is being forwarded to human support.

  The query is then handled manually by a human expert.

- **How Human Response is Integrated**

  After the human provides a response, it can be delivered back to the user.

  This response can also be used to improve the system in future by updating the knowledge base.

6. API / Interface Design

The system provides a simple interface for user interaction.

It allows users to input queries and receive responses generated by the system.

- **Input Format**

  The input to the system is a text-based user query.

  Example: "What is the attendance policy?"

- **Output Format**

  The output is a text-based response generated by the LLM using retrieved context.

  It may also include a message indicating escalation to human support if needed.

- **Interaction Flow**

  1. The user enters a query.
  2. The system processes the query.
  3. Relevant data is retrieved from the vector database.
  4. The LLM generates a response.
  5. The response is displayed to the user or escalated if necessary.

7. Error Handling

Error handling ensures that the system can manage unexpected situations and provide appropriate responses without failure.

- **Missing Data**

  If the input document is missing or cannot be loaded, the system displays an error message and stops execution.

- **No Relevant Chunks Found**

  If the retriever does not find any relevant chunks, the system informs the user that no relevant information is available.

- **LLM Failure**

If the LLM fails to generate a response due to API or processing issues, the system provides a fallback message or retries the request.